

STUDYIA

Plan de Desarrollo

1. Idea general del sistema

StudyIA será una aplicación web para estudiantes que combinará una interfaz web, una API, una base de datos y servicios de Inteligencia Artificial.

Sistema inteligente de asistencia académica mediante IA

Nombre del proyecto:

StudyIA — Asistente Inteligente para Estudiantes

¿De qué trata?

Es una aplicación web que utiliza **Inteligencia Artificial** para ayudar a los estudiantes con sus actividades académicas.

El sistema permitiría:

-  Hacer preguntas a un asistente de IA.
-  Explicar temas difíciles de forma sencilla.
-  Generar resúmenes de textos.
-  Crear preguntas de práctica a partir de un tema.
-  Registrar el progreso del estudiante.
-  Recomendar temas que debería reforzar.
-  Crear cuentas e iniciar sesión.

Tecnologías

Para hacerlo desde cero en **Visual Studio Code**, podemos utilizar: _____

2. Tecnologías principales

Tecnología	Uso dentro de StudyIA
Angular 22	Frontend: pantallas, formularios, dashboard, chat y visualización del progreso.
TypeScript	Lenguaje utilizado en Angular.
Python	Lenguaje del backend y de la lógica relacionada con IA.
FastAPI	Construcción de la API REST que comunica Angular con el backend.
PostgreSQL	Almacenamiento de usuarios, materias, temas, preguntas, respuestas y progreso.
SQLAlchemy	ORM para trabajar con PostgreSQL desde Python.
Pydantic	Validación de datos de entrada y salida en FastAPI.
JWT	Autenticación de usuarios.
LLM	Chat, explicaciones, resúmenes y generación de preguntas.
Git + GitHub	Control de versiones y trabajo colaborativo.
Docker	Opcional; facilitará reproducir el entorno cuando el proyecto esté estable.

3. Fases del desarrollo

El proyecto se debe construir de forma incremental. No conviene comenzar directamente con la IA. Primero se construye la aplicación base y después se incorpora inteligencia.

Fase	Nombre	Qué se hace	Tecnologías	Resultado
FASE 1	Definir el MVP	Decidir exactamente qué tendrá la primera versión.	Ninguna / documentación simple	Lista de funciones y alcance cerrado.
FASE 2	Preparar entorno	Instalar y configurar las herramientas.	Angular 22, Node.js, Python, FastAPI, PostgreSQL, Git	Todos pueden ejecutar el proyecto.
FASE 3	Diseñar base de datos	Definir cómo se guardará la información.	PostgreSQL + SQLAlchemy	Modelo de tablas y relaciones.
FASE 4	Crear backend base	Construir la API y conexión con BD.	Python + FastAPI + SQLAlchemy + Pydantic	API funcionando con Swagger.
FASE 5	Autenticación	Registrar e identificar estudiantes.	FastAPI + JWT + PostgreSQL	Registro/login funcionando.
FASE 6	Crear frontend base	Construir la aplicación visual.	Angular 22 + TypeScript + Angular Material opcional	Pantallas principales funcionando.
FASE 7	Módulo académico	Materias, temas, preguntas y respuestas.	Angular + FastAPI + PostgreSQL	El estudiante puede estudiar y responder.
FASE 8	Progreso	Guardar y mostrar resultados.	FastAPI + PostgreSQL + Angular + gráficos	Dashboard de progreso.
FASE 9	Integrar IA	Añadir las funciones inteligentes.	FastAPI + servicio LLM	Chat, explicación, resumen y generación.
FASE 10	Recomendaciones	Detectar debilidades y recomendar temas.	Python + reglas; ML opcional	Recomendaciones personalizadas.
FASE 11	RAG opcional	Permitir preguntas sobre material académico.	Embeddings + almacenamiento vectorial + LLM	IA basada en documentos.

FASE 12	Pruebas y presentación	Comprobar que todo funciona.	Tests + pruebas manuales + evaluación	Versión final demostrable.
---------	------------------------	------------------------------	---------------------------------------	----------------------------

4 . Detalle de cada fase

FASE 1 — Definir el MVP

- Definir las funciones mínimas: registro/login, materias, temas, preguntas, resultados, progreso, chat IA y recomendaciones.
- Decidir qué funciones quedan para después.
- Definir qué significa que el proyecto esté terminado.

Resultado: Resultado: una lista corta de funcionalidades que los tres integrantes entienden de la misma manera.

FASE 2 — Preparar el entorno

- Instalar Node.js compatible con Angular 22.
- Instalar Angular CLI.
- Instalar Python y crear un entorno virtual.
- Instalar FastAPI y las dependencias del backend.
- Instalar PostgreSQL.
- Crear repositorio Git.
- Definir estructura inicial de frontend y backend.

Resultado: Resultado: los tres integrantes pueden clonar el proyecto y levantarlo en su computadora.

FASE 3 — Diseñar la base de datos

- Crear la base de datos StudyIA.
- Diseñar tablas de usuarios, materias, temas, preguntas, opciones, intentos, progreso y recomendaciones.
- Definir claves primarias y foráneas.
- Definir relaciones.
- Crear modelos SQLAlchemy y migraciones.

Resultado: Tecnologías: PostgreSQL + SQLAlchemy + Alembic (recomendado para migraciones).

FASE 4 — Backend base

- Crear proyecto FastAPI.
- Crear configuración por variables de entorno.
- Conectar PostgreSQL.
- Crear modelos y esquemas.
- Crear endpoints CRUD.
- Probar los endpoints desde Swagger.

Resultado: Resultado: una API funcional antes de incorporar IA.

FASE 5 — Autenticación

- Crear registro de usuarios.
- Hashear contraseñas.
- Crear login.

- Generar JWT.
- Proteger endpoints privados.
- Probar autorización.

Resultado: Resultado: cada estudiante puede acceder solamente a sus datos.

FASE 6 — Frontend Angular 22

- Crear layout principal.
- Crear login y registro.
- Crear menú.
- Crear dashboard.
- Crear páginas de materias y temas.
- Crear servicios Angular para consumir FastAPI.

Resultado: Resultado: frontend conectado al backend.

FASE 7 — Módulo académico

- Mostrar materias.
- Mostrar temas.
- Mostrar preguntas.
- Permitir responder.
- Registrar el intento.
- Mostrar resultado.

Resultado: Resultado: ya existe una aplicación educativa funcional incluso sin IA.

FASE 8 — Progreso

- Calcular porcentaje de aciertos.
- Calcular intentos por tema.
- Identificar temas fuertes y débiles.
- Crear dashboard.
- Mostrar evolución del estudiante.

Resultado: Tecnologías: Python para cálculos + PostgreSQL para datos + Angular para visualización.

FASE 9 — Inteligencia Artificial

- Crear un servicio independiente para IA dentro de FastAPI.
- Elegir proveedor de LLM.
- Crear endpoint de chat.
- Crear función de explicación.
- Crear función de resumen.
- Crear función de generación de preguntas.
- Controlar errores y límites de uso.

Resultado: Resultado: StudyIA deja de ser solamente una aplicación CRUD y comienza a proporcionar asistencia inteligente.

FASE 10 — Recomendaciones

- Usar los resultados almacenados.
- Definir reglas iniciales. Ejemplo: si un tema tiene menos de 60% de aciertos, recomendarlo.
- Generar recomendaciones explicables.

- Mostrar recomendaciones en Angular.
- Opcionalmente comparar posteriormente las reglas con un modelo de Machine Learning.

Resultado: Recomendación: comenzar con reglas. No necesitan Machine Learning desde el primer día.

FASE 11 — RAG (OPCIONAL)

- Permitir cargar material académico.
- Extraer texto de documentos.
- Dividir el texto en fragmentos.
- Crear embeddings.
- Guardar y recuperar fragmentos relevantes.
- Enviar el contexto recuperado al LLM.

Resultado: Resultado: el estudiante puede preguntar basándose en materiales específicos de una materia.

FASE 12 — Pruebas y presentación

- Probar login.
- Probar preguntas y resultados.
- Probar cálculo de progreso.
- Probar respuestas del LLM.
- Probar recomendaciones.
- Corregir errores.
- Preparar datos de demostración.

Resultado: Resultado: una versión estable para presentar.

Explicación en donde se ingresa la IA

La IA no debe controlar todo el sistema. Se recomienda separar las funciones.

Función	Tecnología principal	¿IA?	Explicación
Login	FastAPI + PostgreSQL + JWT	No	Es lógica tradicional.
Materias/temas	FastAPI + PostgreSQL + Angular	No	CRUD normal.
Registrar respuestas	FastAPI + PostgreSQL	No	Datos estructurados.
Calcular porcentaje	Python	No	Algoritmo matemático.
Chat	LLM	Sí	Generación de lenguaje.
Explicar tema	LLM	Sí	Generación adaptada al estudiante.
Resumir texto	LLM	Sí	Procesamiento/generación de texto.
Generar preguntas	LLM	Sí	Generación de contenido.
Detectar debilidad	Python + datos	Inicialmente no	Reglas basadas en resultados.
Recomendar temas	Reglas / ML opcional	Puede ser	Se puede evolucionar a ML.
Responder sobre PDFs	RAG + LLM	Sí	Fase avanzada.

5. ¿Qué modelo de IA necesitamos?

Para comenzar, no necesitamos entrenar un modelo desde cero. La opción más sencilla es utilizar un LLM existente mediante una API y llamarlo desde FastAPI.

Primera versión: LLM por API.

Segunda versión opcional: RAG con documentos académicos.

Versión avanzada opcional: Machine Learning para mejorar las recomendaciones si se dispone de suficientes datos.

6. Repartición

Integrante	Responsabilidad principal	Fases principales
Integrante 1	Frontend	Fases 6, 7, 8 y apoyo en 12
Integrante 2	Backend + BD	Fases 3, 4, 5, 7, 8 y apoyo en 12
Integrante 3	IA + recomendaciones	Fases 9, 10, 11 y apoyo en 12

7. MVP recomendado

Para la primera versión, recomiendo NO intentar hacer todo. El MVP sería:

- Registro e inicio de sesión.
- Materias y temas.
- Preguntas de opción múltiple.
- Registro de respuestas.
- Cálculo de progreso.
- Dashboard.
- Chat con IA.
- Explicación de temas.
- Generación de preguntas.
- Recomendación básica de temas débiles.

8. Funciones que pueden dejar como ampliación

- Carga de PDFs.
- RAG.
- Embeddings.
- Historial avanzado de conversaciones.
- Modelo de Machine Learning para recomendaciones.
- Panel de profesores.
- Notificaciones.
- Aplicación móvil.
- Gamificación.
- Despliegue en la nube.

9. Orden recomendado para empezar

1. Reunirse los tres y aprobar el MVP.
2. Crear repositorio Git.
3. Definir estructura frontend/backend.
4. Preparar Angular 22, Python/FastAPI y PostgreSQL.
5. Diseñar la base de datos.

- 6. Crear backend y conexión a PostgreSQL.
- 7. Crear autenticación.
- 8. Crear frontend.
- 9. Conectar Angular con FastAPI.
- 10. Crear módulo de preguntas y resultados.
- 11. Crear dashboard de progreso.
- 12. Integrar el LLM.
- 13. Crear recomendaciones.
- 14. Evaluar si RAG entra en el tiempo disponible.

10. Resultado esperado al terminar cada gran etapa

Etapas	Debe poder demostrarse
Backend	Swagger muestra endpoints funcionando y datos en PostgreSQL.
Frontend	Angular permite navegar y consumir la API.
Módulo académico	Un estudiante puede responder preguntas y guardar resultados.
Progreso	El sistema muestra estadísticas por tema.
IA	El estudiante conversa, solicita explicaciones y genera preguntas.
Recomendaciones	El sistema identifica debilidades y propone qué estudiar.
RAG	Opcionalmente, el sistema responde usando material académico cargado.
Final	Todo funciona integrado en una sola aplicación.